

GitMini

Software Requirement Specifications

Tanzir Kazi

1. Overview

1.1 Introduction

GitMini is a simplified version control system designed to explore and demonstrate the fundamental ideas behind modern source control tools such as Git. Rather than focusing on optimization or full feature parity, GitMini prioritizes understandability by implementing a minimal set of commands over a content-addressed object store. Through this design, the system exposes how file snapshots, directory structure, and commit history interact to form a complete version history.

1.2 Scope

GitMini will not be offering any networked capabilities: cloning, merging, pulling, etc. GitMini will not include other standard Git features, such as tags, history rewrites, a staging area. GitMini will be contained to the command line, with no graphical user interfaces.

2. Requirements

2.1 *Functional Requirements: The System-to-be Shall...*

2.1.1 *Repository & Objects*

- The system shall create a dedicated metadata directory.
- The system shall store content-addressed hashes and reuse identical objects.

2.1.2 *Committing*

- The system shall allow users to save the current version of their working directory
- The system shall allow users to associate messages or other metadata with each commit

2.1.3 *Checkout*

- The system shall allow users to restore their working directory to a previous commit, using that commit's hash.
- The system shall warn users if checking out will destroy uncommitted changes

2.1.4 *References / Tracking*

- The system shall keep track of the tips of all branches.
- The system will be able to track the current branch (HEAD) and update the branch tip on commit

2.1.5 *History / Visualization*

- The system shall display commit ancestry (log) and branch tips.
- The system shall display a commit graph (version tree) in text form.
- The system shall display object metadata and references (inspect).

2.1.6 *Diff & Similarity*

- The system shall compute structural differences between two commits (which blobs/trees changed).
- The system shall compute an optional content similarity score for changed blobs/commits.

2.1.7 *UX*

- The system shall provide help output for each command and meaningful error messages.
- The system shall be completely usable via command line

2.2 *Non-functional Requirements:*

- Remain human-readable
- Prevent data loss
- Commands will be intuitive

2.3 Data Model

GitMini stores all data in a content-addressed object database. Each object (file contents, directories, and commits) is identified by a cryptographic hash of its contents and stored in `.gitmini/objects`. Before writing an object, GitMini checks whether an object with the same hash already exists, naturally deduplicating identical data. Commits reference tree objects, trees reference blobs and subtrees, and parent pointers link commits into a directed acyclic graph representing version history.

2.4 Commands

<code>gitmini init</code>	- Initializes a new GitMini repository in the current directory - Creates the <code>.gitmini/</code> directory and the required subdirectories for object storage and references
<code>gitmini commit -m "<message>"</code>	- Creates a new commit representing a snapshot of the current working directory - Hashes blobs, creates trees, and stores commit object containing metadata and a parent pointer
<code>gitmini checkout <commit-hash branch></code>	- Restores the working directory to match the snapshot referenced by the given commit hash
<code>gitmini log</code>	- Displays the commit history starting from HEAD
<code>gitmini tree <commit-hash HEAD></code>	- Displays the directory tree associated with the specified commit, including filenames, object types, and hashes
<code>gitmini diff <commitA><commitB></code>	- Compares two commits and reports the added, removed, and modified files
<code>gitmini similarity <commitA><commitB></code>	- Computes how similar (percentage overlap) two commits are based on shared objects.
<code>gitmini branch <name></code>	- Creates a new branch pointing to the current HEAD commit
<code>gitmini cat-object <hash></code>	- Displays the raw contents and type of the specified object
<code>gitmini status</code>	- Displays files changed since last commit

4. Constraints & Assumptions

4.1 Constraints

4.1.1 Local Machine Only

- GitMini will not support any networked capabilities or shared repositories.

4.1.2 No Object Compression

- GitMini will not compress object files.
- Depending on the working directory and timespan, .gitmini folder will grow large.

4.1.3 Single Working Directory

- GitMini will only track the directory where .gitmini was initialized, rather than tracking nested repositories.

4.1.4 Linear History

- No merges allowed. While branches may occur, merging will not be supported.

4.2 Assumptions

4.2.1 Existence of .gitmini/

- All GitMini metadata resides in .gitmini/
- Users do not manually edit files here.

4.2.2 Cryptographic Hash Uniqueness

- Using SHA1, hash collisions will be assumed to be impossible.

4.2.3 Text and Binary Files are Treated Identically

- All files are stored as raw bytes.

5. Future Work

Future versions may support...

- Merges
- Compression
- Tags
- Ignore Rules
- Index + Staging Area
- Minimal Remote Support